



Universidad de Guadalajara
Sistema de Educación Superior
Centro Universitario de Ciencias Exactas e Ingenierías

Arquitectura de Computadoras
Sección D05
Profesor: Jorge Ernesto Lopez Arce Delgado

A04 Sumador Nibble y 8 Bits

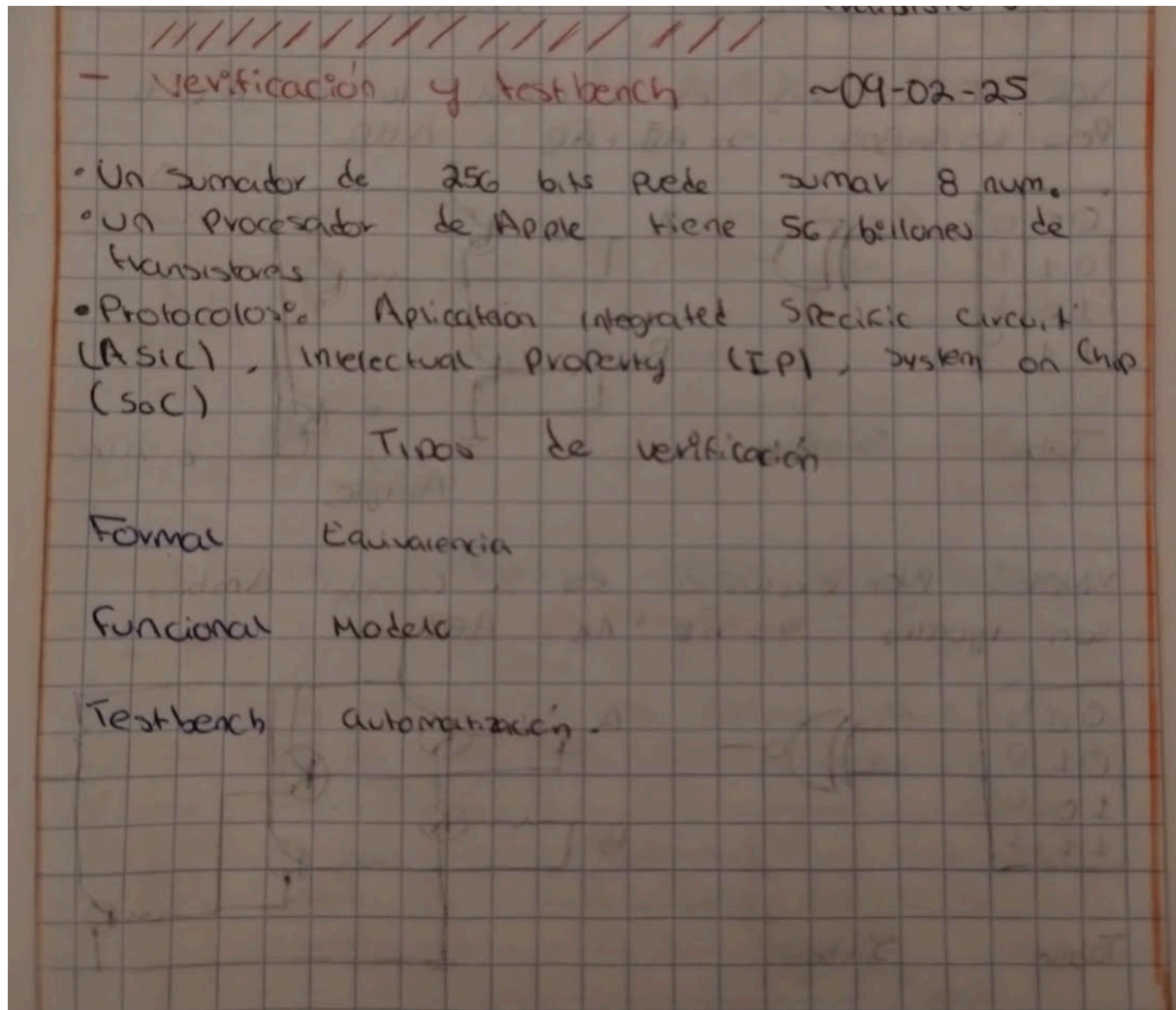
Ingeniería en Computación

Ortiz Segura Emmanuel Alejandro

221501336

15 de Febrero de 2025

INTRODUCCIÓN



Meltdown y Spectre

Descubiertas en 2017, son dos de las vulnerabilidades de hardware más extendidas del mundo, afectan PCs, teléfonos, tabletas, servidores...

Las CPU realizan los procesos de pensar y actuar, Meltdown y Spectre se pueden explotar para que los hackers lean la "mente" y robar datos privados.

Meltdown: En la mayoría de dispositivos los programas y el SO están en "sandbox" o aislados, Meltdown "derriba" los muros y afecta a las CPU de Intel, Apple y algunas de IBM y ARM.

Spectre: Toma su nombre de speculative execution y engaña procesadores para que los ejecuten comandos fuera de su alcance, es más difícil explotar y más difícil de parchear. Afecta a las CPU de Intel, Apple, ARM, AMD.

Hasta el momento existen 13 variantes de Spectre y 14 de Meltdown.

Fueron descubiertas por investigadores de Project Zero (Google) y algunas universidades.

Ejecución especulativa y ejecución fuera de orden

Sumador Mibble y Sumador 8 bits

Conceptos importantes:

- Complemento a 1° Sirve para manipular números negativos

Reglas

- Cambiar 1's por 0's y 0's por 1
- El bit más significativo indica el signo
0 → + 1 → -

- Complemento a 2° Proviene de la fórmula
 $Cb(N) = b^n - N$ $b = \text{base}$, $n = \text{anchura digital}$

Truco: Para NO HACER LA RESTA

1. hasta el primer 1 los números van a coincidir
2. los demás se invierten

Ejemplo: $n=8 \rightarrow C_2(11001010) = 00110110$

Explicación

Se invierten
Va a sobrar el bit más significativo al sumar el número base y el complemento, al quitarlo da 000... por lo que el complemento sirve como un $-N$

Punto Flotante: representaciones internas del Procesador que modelan los números reales.
como 45.12×10^{-5} a 9.512×10^{-4}

Se compone en:

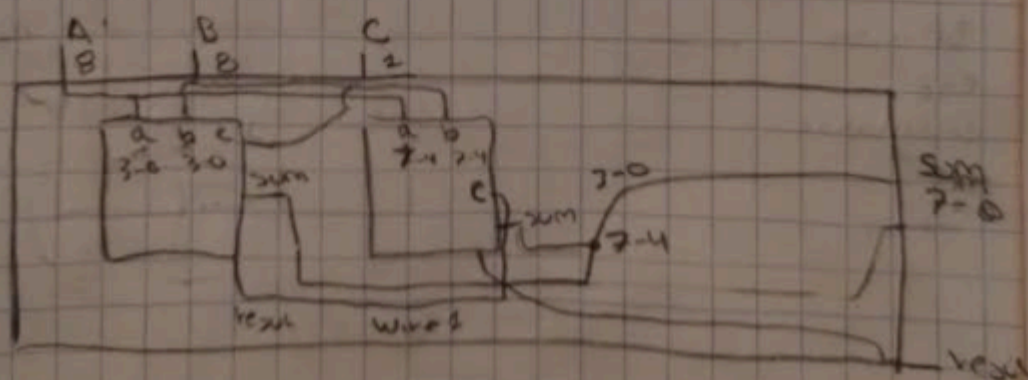
Mantisa $\rightarrow$ corresponde a la parte fraccionaria que es la diferencia entre el número y la parte interna entera (cuando es negativo es su complemento)

Exponente $\rightarrow$ Indica dónde se coloca el punto decimal en relación al inicio de la mantisa.

ESTÁNDAR IEEE 754.

- **Presión simple:** 32 bits, bits significativos = 23 + 1 signo bits del exponente 8. número más pequeño $\sim 1.28 \times 10^{-38}$ número más grande $\sim 3.4 \times 10^{38}$
- **Presión doble:** 64 bits, bit Significativo 52 + 1 signo bits del exponente 11, n.m. pequeño $\sim 5.0 \times 10^{-324}$ y más grande $\sim 1.08 \times 10^{308}$

Sumador 8 bit:



OBJETIVOS:

Entender cómo funciona la suma desde el diseño de hardware, desde el medio sumador, sumador completo, sumador de 4 bits y sumador de 8 bits.

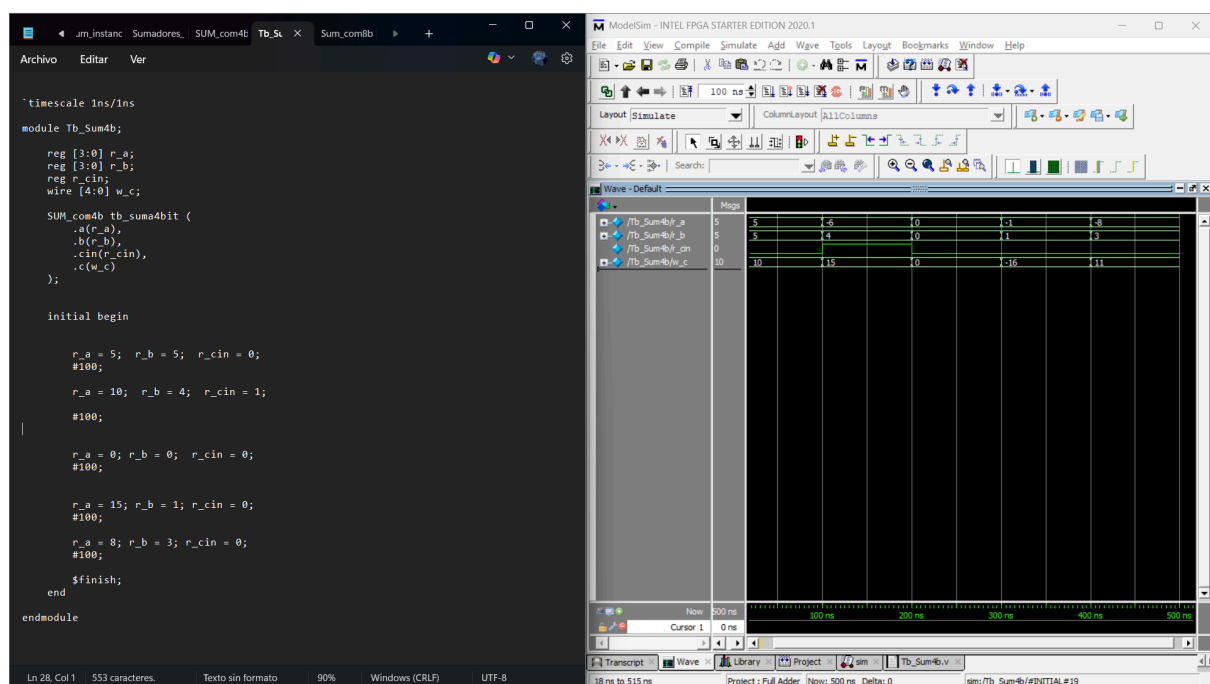
Comprender y diseñar estos sumadores en el lenguaje de descripción Verilog.

Comparar las diferencias entre un sumador estructural (con instancias) y un sumador comportamental.

Diseñar un test benches para comprobar que el diseño del hardware sea correcto implementando 5 sumas para el de 4 bits comportamental y otras 5 para el de 8.

Analizar la simulación y el wave del diseño en formato decimal.

DESARROLLO



Como se puede observar a la derecha del código, el wave muestra 5 sumas diferentes:

Los valores se muestran en un radix decimal configurado en el propio programa de modelsim para facilitar la lectura.

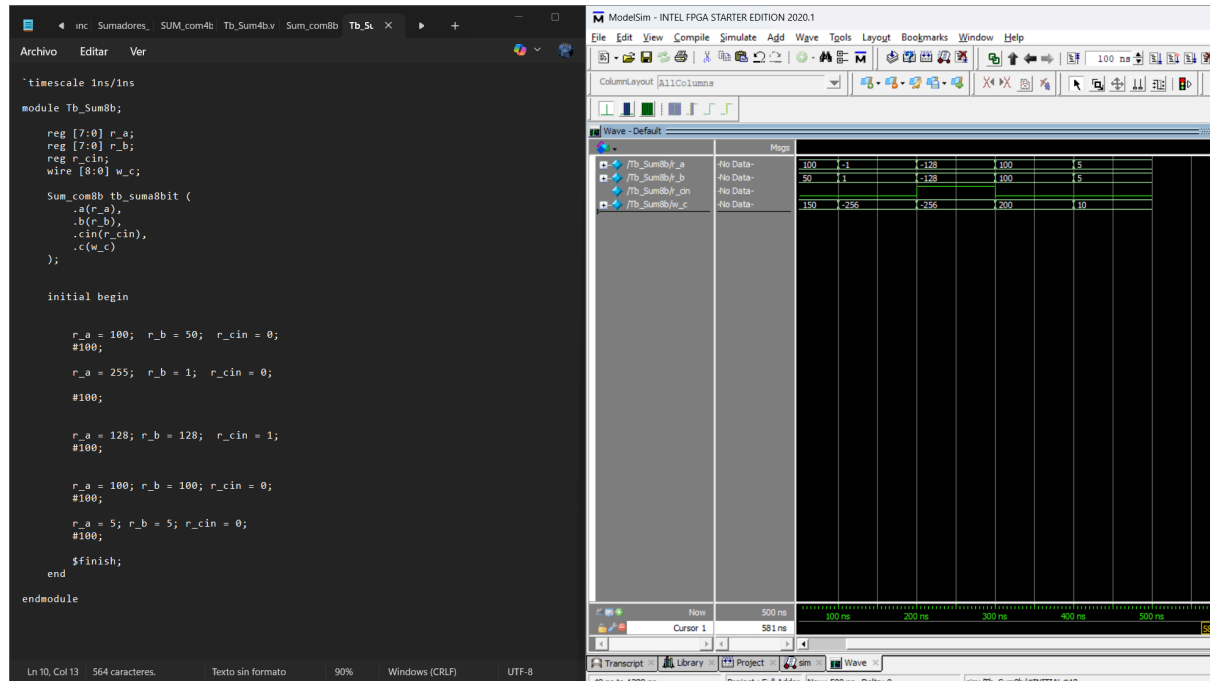
La primer suma es correcta 5+5 es 10

La segunda suma $10 + 4$ con uno de acarreo inicial es 15 aunque en el wave se muestra como $-6 + 4$ pero el resultado esperado es el correcto.

La tercera suma es $0+0 = 0$ lo cual es correcto

La cuarta suma es una suma que excede los 4 bits $15 + 1 = 16$ por eso su comportamiento es inusual

Y por último $8 + 3 = 11$ también es correcta por lo que la primer prueba fue exitosa



Aquí de igual manera analizando el wave se puede observar que:

la primer suma es correcta $100+50=150$

La segunda suma también es correcta $255+1=256$

La tercera suma tiene un ligero error ya que $128 + 128 + 1$ de acarreo debería dar 257 así que sería analizar la falla.

la cuarta suma es correcta $100 + 100$ es 200

y la ultima suma también es correcta $5+5=10$

Concluimos que en el diseño hay un ligero error al momento de tomar el acarreo.

CONCLUSIONES

Durante la práctica desarrollé diferentes habilidades que necesitaba para llevarla a cabo.

Para poder diseñar los sumadores necesité aprender cómo se declaran cables y para poder pasar los bits, desde la sintaxis hasta imaginar y dibujar cómo funciona el diseño ya que de lo contrario si primero lo quería programar tenía muchos errores de lógica y sintaxis.

Además tuve que diferenciar entre entradas, salidas y buses de datos ya que una salida podría ser la entrada de otro módulo pero esa entrada de módulo tenía un cable que se dividía en 2 y cada uno pasaba diferentes datos del bit más significativo al menos.

Al instanciar los módulos pasa al revés, ya que al realizar la operación se hace desde el menos significativo al más significativo como en el álgebra convencional.

Para probar que todo esté correcto también desarrolle un par de habilidades como el uso del testbench.

El testbench es un módulo donde adentro metes el diseño a probar, también llamada Unit Under Test (UUT). El testbench no tiene inputs o outputs pero contiene registros que son los datos de entrada por decirlo de alguna manera y los wires, que son los cables que nosotros vemos.

Para realizar un test bench me basé en un estándar que investigué al que llame [D.I.GE](#)

1.Declarar: Aquí declaro el módulo, los registros y los wires de acuerdo a la unidad bajo prueba.

2.Instanciar: Aquí hago la instancia de la UUT

3. Generación de estímulos: esta es la parte más compleja, aquí me basé en algunos códigos de internet para comprender el funcionamiento.

Primero se inicializa con initial begin, luego se le dan los valores a cada registro y se establece algún tiempo por ejemplo 100 ns (#100) y finalmente se termina la condición, y el módulo con \$finish, end, endmodule

REFERENCIAS

Jiménez Murillo, J. A. (n.d.). SISTEMAS NUMÉRICOS: SUMA DE DOS CANTIDADES EN COMPLEMENTO a 2. In Matemáticas para la computación.
https://libroweb.alfaomega.com.mx/book/681/free/ovas_statics/cap1/Ejerciciosdecomplementados.pdf

Cisneros Martínez, J. A. (n.d.). Complemento a 1 y 2 de números binarios.
<https://pvjl.pbworks.com/f/complemento%20a%202.pdf>

Organización de Computadoras. (n.d.). Organización de Computadoras.
<https://cs.uns.edu.ar/~ldm/mypage/data/oc/apuntes/2019-modulo5.pdf>

(Libro en PDF) Principios de arquitectura de computadoras: https://academicos.azc.uam.mx/oan/apc/Murdocca_es.pdf

IAY0340-Digital Systems Modeling and Synthesis. (n.d.).
<https://ati.ttu.ee/IAS0340/labs/Tutorials/SystemC/Adder.html>